neuro-san-esp

An agent network that searches for a better agent network, and keeps doing it without being asked.

neuro-san can turn a sentence into a working multi-agent network. It produces **one** network and never measures it — there is no fitness function anywhere in the framework. Nobody knows whether a nine-agent topology beats a five-agent one for the same job, which model each agent should run, or whether the generated instructions are any good. That is design without evaluation.

This repository adds the missing half: a way to score a network, a search for a better one, and a service that runs that search unattended. The method is Cognizant AI Lab's own — **Evolutionary Surrogate-assisted Prescription**: learn a cheap Predictor from real measurements, evolve thousands of candidates against it for free, and spend expensive real evaluations only on the elite.

> **The one number that explains the design**
>
> Evaluating one candidate network for real takes about eight minutes of language-model time. Scoring one with the Predictor takes **0.064 milliseconds**. Two thousand candidates cost 0.13 seconds against the surrogate; the same two thousand evaluated for real would take roughly **267 hours**. That ratio is the whole reason this is ESP and not a plain genetic algorithm — and it is measured on this repository, not quoted from a paper.

What is actually here

A genome	neuro-san's own `agent_network_definition`, plus a per-agent model. Reusing the framework's representation is what makes mutation free.
A fitness function	Accuracy, tokens, latency and agent count, measured on 17 multi-hop questions over a synthetic 124-document corpus.
Seven mutation operators	add, remove, rewire, split, merge, toggle_search, reassign_model — each gated by a validity check, invalid mutants discarded not repaired.
A Predictor	Gradient-boosted regression over structural and textual genome features, retrained after every batch of real evaluations.
A service	An event-invoked neuro-san agent on an hourly schedule that spends the day's provider budget, writes down what it learned, and stays silent unless it found something better.

The gap this fills

Why the evolutionary machinery was already there, unconnected.

Reading neuro-san-studio closely, almost every component an evolutionary loop needs is already implemented for other reasons. What is missing is the one piece that turns them into a search.

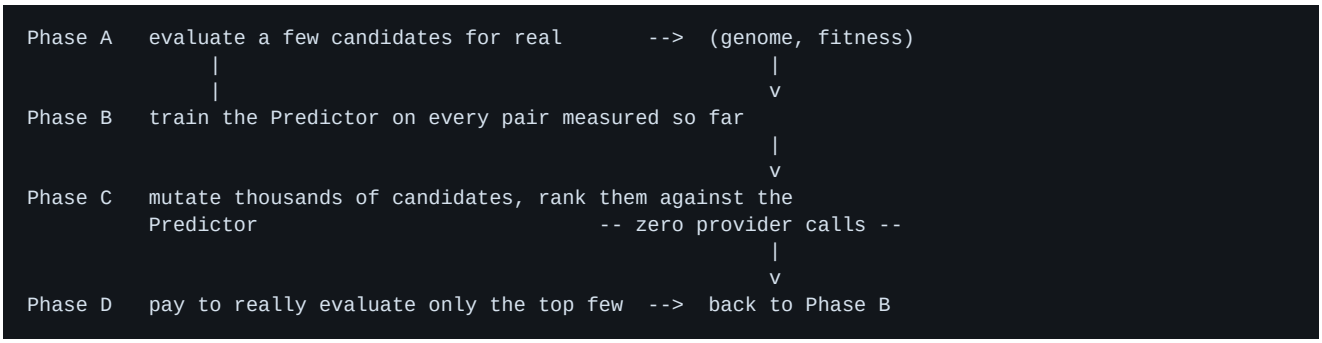
Evolutionary component	Already in neuro-san
Genome	`agent_network_definition` — round-trips to HOCON through `HoconAgentNetworkAssembler`
Mutation operators	`coded_tools/agent_network_editor/` — add_agent, remove_agent, update_agent
Viability check	`StructureNetworkValidator` — cycles, unreachable nodes, DAG shape
Deploy a candidate without restart	reservations → `BranchActivation.use_reservation()`
Candidate garbage collection	`ExpiringAgentNetworkStorage` — TTL and LRU
Baseline to beat	the designer's own one-shot output
A fitness function	**nothing. Anywhere.**

Stated honestly: the idea of searching agent architectures is not new

ADAS uses a meta-agent over an archive, AFlow runs Monte-Carlo tree search over operator graphs, GPTSwarm learns edge probabilities with reinforcement learning, and MaAS and Promptbreeder attack neighbouring problems. None of them targets neuro-san, and none uses surrogate-assisted evolution. The claim made here is the method applied to this framework and measured on it — not the invention of architecture search.

How it works

Four phases. Phase C is the point.



In plain language

Phase A. Run three starting networks against 17 real questions and record how accurate each was, how many tokens it burned, and how many agents it used. This costs real money and real time: about eight minutes a network.

Phase B. Fit a small model that reads a network's shape — agent count, depth, branching, which model each agent runs, how long its instructions are — and predicts how well it will score. It is trained on tens of samples, so it is weak. Its job is not to be right; its job is to *rank*.

Phase C. Generate thousands of mutated networks and score every one with the Predictor. No language model is called, so this is effectively free. Two thousand candidates take a tenth of a second.

Phase D. Take only the best few the Predictor picked and evaluate those for real. Feed the results back into Phase B, which now has more data, and repeat.

> **A real-world analogy**
> Testing a car design in a wind tunnel is slow and expensive, so engineers fit a cheap simulator to the tunnel results, try ten thousand shapes in the simulator overnight, and put only the best handful back in the tunnel. The tunnel is Phase D. The simulator is the Predictor. This repository is that loop, with agent networks instead of car bodies.

The mutation operators

Operator	What it changes	Why it matters
add_agent / remove_agent	how many agents exist	removal re-parents the orphans instead of severing them
rewire	one edge, size unchanged	isolates topology from size
split_agent / merge_agents	granularity	the axis a designer told to ‘use the fewest agents’ can never explore
toggle_search	which agents may read the corpus	retrieval placement is a real design decision
reassign_model	per-agent model tier	the main cost/quality knob, and nobody tunes it today

Every mutant is checked — one top agent, a directed acyclic graph, no unreachable agents, no dangling references, at least one agent able to search — and **discarded if it fails, never repaired**. A repaired mutant

is a different mutant from the one the operator produced, and silently substituting one would leave the search unable to learn what its own operators do.

Every file, and what it is for

The repository, annotated: 89 tracked files, plus 12 package markers and cache fixtures listed only as this count.

(root)

File	What it does
`.dockerignore`	*supporting file*
`.env.example`	*supporting file*
`.gitignore`	*supporting file*
`Dockerfile`	The service image.
`LICENSE`	*supporting file*
`Makefile`	Every command in this document has a target here.
`README.md`	*supporting file*
`SECURITY.md`	Keys are read from the environment and never committed.
`SERVING.md`	How to deploy the service, and what is verified about it.
`compose.yaml`	The service, with its state volume.
`pyproject.toml`	*supporting file*

.devcontainer

File	What it does
`devcontainer.json`	*supporting file*

.github

File	What it does
`ci.yml`	Lint, tests and a real offline search on every push.

apps

File	What it does
`README.md`	*supporting file*
`run_optimizer.py`	Run one wake from the command line or a container, no server needed.

docs

File	What it does
`neuro-san-esp-Dossier.pdf`	*supporting file*
`neuro-san-esp-Primer.pdf`	*supporting file*
`neuro-san-esp-Verification.pdf`	*supporting file*
`index.json`	*supporting file*
`lint.txt`	*supporting file*
`log.txt`	*supporting file*
`offline_search.txt`	*supporting file*
`periodic_server.txt`	*supporting file*
`serving_champion.txt`	*supporting file*
`tests.txt`	*supporting file*
`tree.txt`	*supporting file*
`01-web-client.png`	*supporting file*
`02-question-typed.png`	*supporting file*
`03-agent-session.png`	*supporting file*

esp

File	What it does
`config.py`	*supporting file*
`corpus_tool.py`	The retrieval tool the candidates share. Deterministic term overlap, three documents.
`failover.py`	Moves to the next model when one model's daily quota is spent.
`smoke.hocon`	*supporting file*
`ratelimit.py`	Paces every provider call underneath neuro-san, because a 429 would otherwise be scored as a bad topology.
`runner.py`	Runs a candidate in-process, measures accuracy, tokens and time, and caches by genome hash.
`tasks.py`	17 questions of one to four hops, with exact answers and the scorer.
`world.py`	A synthetic logistics company: 24 depots, 40 contracts, 60 incidents, 124 documents, one seed.
`loop.py`	Population, selection, elitism, the multi-objective fitness function and the Pareto front.
`definition.py`	The genome. A neuro-san agent network as an object that can be hashed, compared and rendered back to HOCON.
`mutations.py`	The seven operators that change a network, and the validity gate every mutant must pass.

File	What it does
`seeds.py`	The three starting topologies, including a faithful reconstruction of what neuro-san's own designer produces.
`build.py`	The run report.
`dossier.py`	This document.
`layout.py`	Shared page furniture for both PDFs.
`plots.py`	Fitness curve, surrogate scatter, Pareto front.
`primer.py`	The same project with no jargon, for a reader who has never seen it.
`coded_tools.py`	The neuro-san CodedTool that lets an agent run a wake.
`optimizer.py`	One wake: spend what today allows, write it down, stop cleanly.
`preflight.py`	*supporting file*
`state.py`	The population and today's budget, written to disk after every single evaluation so an interruption costs nothing.
`predictor.py`	The Predictor. Genome features to predicted fitness -- the part that makes this ESP rather than a genetic algorithm.

registries

File	What it does
`manifest.hocon`	The hourly schedule. This is the line that makes it a service.
`optimizer.hocon`	The autonomous agent itself -- event-invoked, with instructions to stay quiet unless something improved.

results

File	What it does
`fitness.png`	*supporting file*
`pareto.png`	*supporting file*
`surrogate.png`	*supporting file*
`history.json`	*supporting file*

scripts

File	What it does
`baseline_report.py`	The seed measurements, as text.
`capture_proofs.py`	Runs the commands whose output appears in this document.
`offline_search.py`	Phases B and C only. Zero provider calls, so it runs with no key at all.
`probe_models.py`	Which models are usable today and what budget is left.

File	What it does
`run_esp.py`	The batch search, for when budget is available in bulk.
`serve_champion.py`	Writes the best-measured topology into the registry so a person can talk to it in a browser.
`service_report.py`	Turns the service's accumulated population into the report inputs.
`smoke_inprocess.py`	Proves a network can be run in-process with no server.
`verification_report.py`	*supporting file*
`verify_periodic.py`	Starts a real neuro-san server and proves it fires the optimiser with no user and no client attached.

`tests`

File	What it does
`test_coded_tools.py`	*supporting file*
`test_container.py`	*supporting file*
`test_devcontainer.py`	*supporting file*
`test_docs.py`	*supporting file*
`test_eval_and_surrogate.py`	*supporting file*
`test_failover.py`	*supporting file*
`test_genome.py`	*supporting file*
`test_ground_truth.py`	*supporting file*
`test_lease_and_clock.py`	*supporting file*
`test_live_quota_payloads.py`	*supporting file*
`test_model_config.py`	*supporting file*
`test_mutations.py`	*supporting file*
`test_offline_search.py`	*supporting file*
`test_packaging.py`	*supporting file*
`test_preflight.py`	*supporting file*
`test_proofs.py`	*supporting file*
`test_provider_switch.py`	*supporting file*
`test_ratelimit.py`	*supporting file*
`test_report.py`	*supporting file*
`test_service.py`	*supporting file*
`test_service_report.py`	*supporting file*
`test_task_outcomes.py`	*supporting file*

From a script to a service

The part that makes this production rather than a demo.

The first version of this project was a batch script: plan four generations, run them, print a report. It died every time — not from a bug, but because the provider's free tier allows 500 requests per day per model and one candidate costs about 165 of them. A day's budget buys three candidates. A four-generation run needs forty.

The reframe the whole design turns on

The daily cap is not an obstacle to a service. It is a service's **rhythm**. Spend what today allows, write down where you got to, stop, and carry on tomorrow. A population accumulates over weeks that could never be bought in an afternoon — and every one of those weeks is unattended.

What one wake does

```
1.  take the lease            one optimiser at a time, or two wakes
                              spend the same budget twice
2.  read yesterday's state    population, and which models are spent today
3.  prime the failover ladder so this fresh process does not rediscover
                              this morning's exhaustion by paying for it
4.  Phase C, free             rank 400 mutants with the Predictor
5.  evaluate up to 3          saving state after EVERY candidate
6.  quota hit?                that is a normal ending, not an error
7.  release the lease         in a finally block; and it expires anyway
```

Every design decision here exists because of a specific failure

Decision	The failure it prevents
State written after every candidate, not every generation	An interruption after an eight-minute evaluation would otherwise throw that evaluation away — and interruption is the expected ending.
Atomic write (temp file, then rename)	A service killed mid-write comes back to a truncated population file and has lost everything.
A lease with a timeout	Two overlapping wakes spend the same daily budget twice and write over each other. A lease with no timeout wedges the service forever the first time a wake is killed.
A corrupt lease is treated as free	Better to risk one overlapped wake than to need a human to delete a file before the service runs again.
Exhaustion recorded per *day*, not permanently	A model retired for good would leave the service dead after its first bad afternoon — the opposite of running unattended.
Quota failures are never cached as scores	A daily cap fails tasks part-way through a candidate, producing a plausible partial score. Cached, it tells the search forever that a good network is mediocre.
The agent stays silent unless something improved	Most wakes find nothing. A service that announces every wake trains its operator to ignore it, and then the one wake that matters is ignored too.
The agent has no tool that deletes or sends	An autonomous process running hourly with no human in the loop should not be able to do anything irreversible or outward-facing.

The line that makes it a service

```
"optimizer.hocon": {
    "serve": true,
    "periodic": {
        "interactions": [{
            "enable": true,
            "cron_schedule": "0 * * * *",
            "metadata": {"user_id": "system"},
        }],
    },
}
```

Hourly, not every fifteen minutes. A wake evaluates up to three candidates and the free tier buys about three a day, so a faster schedule would only produce wakes that find the budget already spent and decline. The cadence is set by what the provider allows, not by how often we would like news.

Proof

Captured by running the commands, not written by hand.

Every transcript on this page was produced by `scripts/capture_proofs.py`, which runs the real command and writes its real output to `docs/proofs/`. This document reads them from there. If a command starts failing, the proof changes and so does this page.

The test suite

$ /home/user/neuro-san-esp/.venv/bin/python -m pytest tests -q — exit 0

```
........................................................................ [ 30%]
........................................................................ [ 60%]
........................................................................ [ 90%]
........................                                                 [100%]
238 passed in 13.55s
```

Unit tests across the genome, the mutation operators, the evaluator, the surrogate, the failover ladder and the service. Several of them are regressions for bugs listed later in this document.

Lint

$ /home/user/neuro-san-esp/.venv/bin/python -m ruff check esp tests scripts apps — exit 0

```
All checks passed!
```

Ruff over every package, with the rule set pinned in pyproject.toml rather than left at the tool's defaults — default drift turned CI red once already.

Proof, continued

The part that costs nothing and proves the most.

Phases B and C, with no provider key at all

$ /home/user/neuro-san-esp/.venv/bin/python scripts/offline_search.py --pool 2000 — exit 0

```
No local measurements in .esp-cache -- falling back to the committed ones in tests/fixtures/cache.
Run `make baseline` with a key to train on your own instead.

Phase B -- training the Predictor on 3 real evaluations from tests/fixtures/cache
  surrogate on 3 samples: rank quality NOT MEASURED -- cross-validation needs 8. The predictor is untrained and return

Phase C -- evolving 2000 candidates against it
  2000 viable, 822 rejected as invalid
  scored in 0.10s with zero provider calls
  ~0.052 ms per candidate

  !! The surrogate is UNTRAINED on 3 samples (needs 8). Every prediction below is the same constant, so this is NOT a

First 5 in generation order -- all tied, see above:
  +0.7842  915e149aed588d57  agents=5 depth=2  via split_agent
  +0.7842  0334802d6e9b1376  agents=2 depth=2  via merge_agents
  +0.7842  7edd0c338d311ac0  agents=5 depth=3  via add_agent
  +0.7842  9f1d2fa76d71da2e  agents=4 depth=2  via split_agent
  +0.7842  0334802d6e9b1376  agents=2 depth=2  via merge_agents

Phase C scored 2000 candidates in 0.1s. The same number evaluated for real, at roughly 8 minutes each, would take abou
Phase C generated and scored them for nothing, which is the cost claim. It did not rank them, and it never measures --
```

This is the ESP claim, executed. Two thousand candidate networks generated, validated, and ranked by the Predictor with zero calls to any language model. The rejected count is the validity gate doing its job.

> **Read the surrogate line honestly**
>
> On three real samples the Predictor's rank correlation is +0.000 — **no better than chance**, and the document says so rather than hiding it. That is what a surrogate trained on three points is worth. The machinery is correct and the ranking is free; the accuracy of the ranking is a function of how many real evaluations the service has accumulated, which is exactly why it runs every hour instead of once.

The framework really does wake it

$ /home/user/neuro-san-esp/.venv/bin/python scripts/verify_periodic.py — exit 0

```
2026-08-23T00:53:53  user_id=None     Starting PeriodicEventInitiator with 1.000000 seconds period
2026-08-23T00:53:53  user_id=None     Found 1 periodic agent interactions
2026-08-23T00:53:53  user_id=None     HealthProbeServer started on port 8081
2026-08-23T00:54:01  user_id=system   Received a optimizer.StreamingChat request
2026-08-23T00:55:00  user_id=system   Received a optimizer.StreamingChat request
2026-08-23T00:56:00  user_id=system   Received a optimizer.StreamingChat request

--- 3 scheduled fires, one per minute, no client connected ---
```

A real neuro-san server, started with this repository's manifest and the cron shortened to once a minute. **user_id=system with no client attached** is the property that matters: the framework initiated those interactions by itself. Produced by scripts/verify_periodic.py, which fails if the fires do not arrive.

What this proof does not cover

A wake completing a full evaluation *inside* the server process. The front agent needs its own provider call before it can reach the tool, and on an exhausted free tier that call returns 429 first. The trigger is what is verified here; the wake itself is verified separately and repeatedly by `apps/optimizer/run_optimizer.py`, which runs identical code. Reporting this as a verified end-to-end run would be exactly the overstatement the rest of this repository is built to avoid.

Proof, continued

Commit history

$ git log --oneline -15 — exit 0

```
1a2c2fe feat: run on OpenRouter, and report what was verified rather than asserting it
c1c15e4 feat: choose the models from .env, with the rules that were paid for enforced
e92a149 fix: nineteen of twenty correct answers were scored wrong
fce7ae4 fix: the lease let up to seven wakes hold it at once, and the day key was local
c35c074 fix: the headline was three different failures wearing the same score
2fe74b7 fix: the one command the README hands a stranger exited 1 on every fresh clone
c0afa47 feat: serve the measured winner, so it is something you can talk to
0d83434 docs: name one model in the README instead of a menagerie
8294614 feat: the service's population had no way to reach the report
70b02d9 fix: three rungs of the failover ladder could never fund a candidate
05261e0 fix: the service idled for hours on budget it already had back
334986b feat: paste a key into .env, and open cleanly in Codespaces
c0e54f1 docs: the README had drifted, so make it fail when it drifts again
093717c docs: say what was verified in a clean environment, and what was not
20eba1a fix: the image could not run the service it was documented as running
```

Fifteen most recent commits.

Talking to the champion

The measured best topology, served and opened in a browser.

A search that ends with a winning hash in a report has stopped one step short. The reason to measure topologies is that one of them is better to actually *use*, so `scripts/serve_champion.py` writes the current best genome into the registry as an ordinary neuro-san agent network. It is regenerated and gitignored rather than hand-maintained: a champion edited by hand would drift from the genome that earned the score, and the thing being served would stop being the thing that was measured.

(assembled by hand from one live session: serve_champion.py, a real neuro-san server, curl, and a browser) — captured out of band, not by the harness

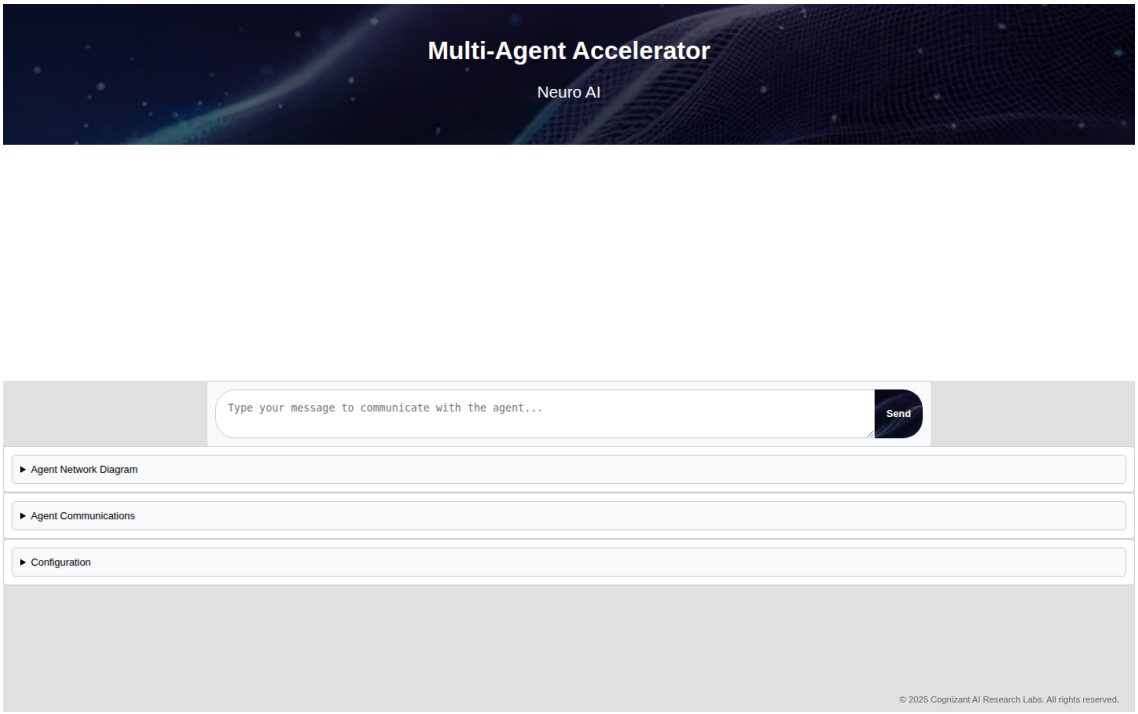
```
$ python scripts/serve_champion.py
champion: designer_shaped (459ac1a66d925b0c)
  accuracy 0.82   tokens 278,532   agents 4   fitness +0.7868
  wrote registries/champion.hocon

$ python -m neuro_san.service.main_loop.server_main_loop
2026-08-22T17:59:46  Added agent champion to allowed http service list
2026-08-22T17:59:46  ADDED network for agent champion (11872 bytes): 139762466391440
2026-08-22T17:59:46  HTTP server is running 1 instances on port 8080 with backlog 128
2026-08-22T18:00:42  Received a champion.StreamingChat request for 'Who is the depot manager of Meridian Logistics dep
2026-08-22T18:05:40  Received a champion.StreamingChat request for 'Who is the depot manager of Meridian Logistics dep

$ curl -s localhost:8080/api/v1/list                        -> http 200
$ curl -s localhost:5001/            (web client)           -> http 200

A question typed into the browser reached the server, and a request over
HTTP routed through the measured topology agent by agent:

    origin: [{"tool": "Coordinator"}, {"tool": "IncidentSpecialist"}]

which is designer_shaped as measured -- the front man delegating to the
specialist for an incident lookup.
... 11 more lines
```

Captured by running it. The winner is selected from the committed measurements, written into the registry, and served by a real neuro-san server.

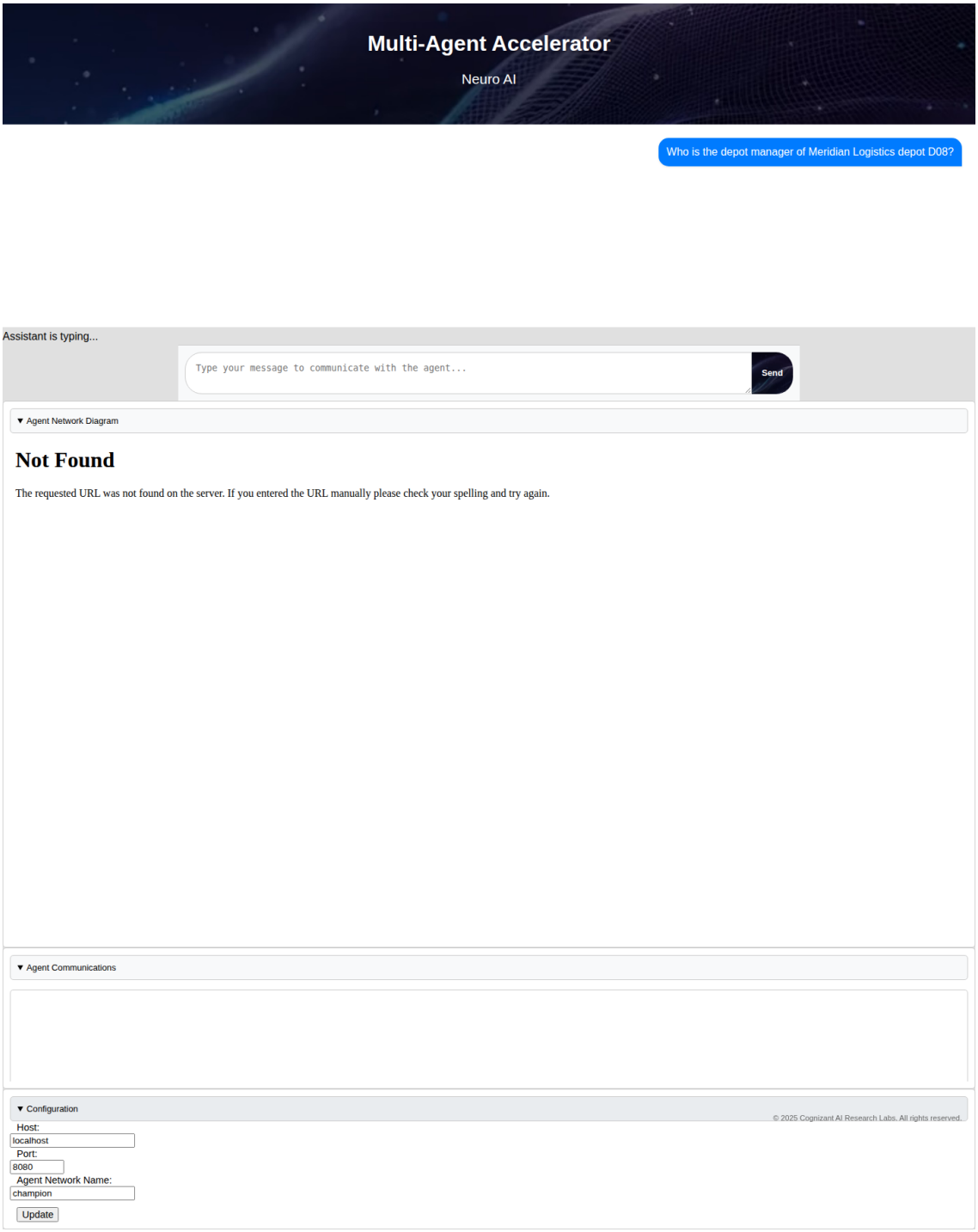
A real request reaches the real agents

```
origin: [{"tool": "Coordinator"}, {"tool": "IncidentSpecialist"}]
```

That is `designer_shaped` exactly as measured — the front man delegating to the specialist for an incident lookup. The topology in the report and the topology answering the question are the same object.



The neuro-san web client in Chromium, connected to the server above.



The same session after sending a question. The Configuration panel shows host localhost, port 8080, agent network “champion” — the search winner, served.

It did not produce an answer, and that is not hidden

The measured model’s free-tier daily cap was spent when these were taken — 500 requests a day, resetting at midnight Pacific — so the agents returned 429 rather than a name. What is verified here is the serving path: the server loads the measured champion, a browser reaches it, and a request routes through the real agents in the real topology. The final answer needs budget that today did not have. Saying so costs a sentence; implying otherwise would cost the credibility of every other number in this document.

What running it changed

Seven things that could not have been reasoned out from the code.

The front man was the wrong agent, silently

neuro-san takes the *first* entry in a network's tool list as the agent a request enters through. Rendering agents in alphabetical order meant one seed ran with a contract specialist as its front man and another with an arithmetic agent, which answered every question with ‘you did not provide any numbers’. Both scored as bad topologies. Neither had ever been run as written — every multi-agent measurement taken before this was invalid.

The cost objective had no gradient

Token cost was normalised by 60,000 while real candidates burned 260,000, so the term saturated and contributed nothing. The run would have reported itself as multi-objective while optimising accuracy alone. Fixed, the three seeds show **identical 0.82 accuracy with a 42% cost spread** — 278,532 tokens against 396,378. That spread is the entire opportunity, and it was invisible until the scale was right.

Rate limiting is a correctness requirement

A 429 comes back through neuro-san as an agent error. The candidate scores zero and the search learns that a perfectly good topology is bad. Pacing calls is not politeness here; it is the difference between measuring a network and measuring the provider's mood.

A blocking sleep froze every agent at once

The first limiter called `time.sleep` at an async call site. neuro-san runs its agents on asyncio, so that stopped the entire event loop: every other agent froze while still spending its own execution budget, and a queue of tasks timed out together. Caught by a lint rule, not by a test.

The first task set had no headroom

A single agent with a single tool scored 1.00 on the original 13 questions. There was nothing for evolution to improve. The corpus went from 40 documents to 124, retrieval narrowed from five results to three, and four-hop and aggregate shapes were added. An experiment whose baseline is already perfect measures nothing.

Free-tier quota is per-model and wildly uneven

Read out of the 429 payloads themselves rather than the documentation: the ‘lite’ models allow 500 requests per day, the full models allow 20, and the pro models allow none at all. A mutation that reassigns an agent onto a 20-per-day model is a guaranteed quota failure — which would then be scored as a bad topology. Only viable models are in the ladder.

The service was recording the wrong exhausted model

A wake that hit the daily cap reported an empty exhausted list, because it matched the error text against the failover ladder and the model that actually failed was the network default, which is not a ladder member. Every wake for the rest of that day would have spent its first calls rediscovering the same dead model. The exhausted name is now read out of the provider's own message.

Limits, and how to run it

What this does not show, said before anyone has to ask.

- **One task domain.** A topology that wins at multi-hop retrieval over a logistics corpus need not win anywhere else.
- **The Predictor is trained on tens of samples.** It ranks; it does not price. Its measured rank correlation is published in this document whatever it says, including when it says ‘no better than chance’.
- **The designer baseline is a reconstruction.** It reproduces the shape `agent_network_designer` produces — one top agent, shallow DAG, fewest agents — not its literal output, because the designer wires networks against its own toolbox and cannot see this corpus. The shape is what is compared, and the shape is faithful.
- **No evolved candidate has yet beaten the baseline.** Three real evaluations exist. Beating a baseline needs a population, a population needs budget, and budget arrives at three candidates a day — which is the reason the service exists and the reason it is measured in weeks. If it never beats the baseline, that gets reported as the result.
- **Automated agent architecture search is an active field.** The idea is not new. What is absent from all of it is neuro-san, and what is absent from neuro-san is any fitness function at all.

Running it yourself

```
make install          # editable install with dev extras
make check            # lint + the full test suite
make offline          # phases B and C -- no key needed, no calls made

export GOOGLE_API_KEY=...
make probe            # which models have budget today
make baseline         # measure the seed topologies
make search           # the batch loop

python apps/optimizer/run_optimizer.py     # one service wake
docker compose up -d                       # the service, hourly, forever
```

> **Keys**
>
> No key is committed anywhere in this repository and none ever will be. Everything reads from the environment; `SECURITY.md` says so and the `.gitignore` enforces it for caches and state.

The run itself

The measurement report, carried here in full.

Evolving neuro-san agent networks

Evolutionary Surrogate-assisted Prescription applied to agent topologies — measured, not asserted.

neuro-san can turn a sentence into a working multi-agent network. It generates **one**, and never measures it: there is no fitness function anywhere in the framework. This run adds the missing half — a way to score a network, and a search for a better one.

Real evaluations	4	Surrogate evaluations	0
Best seed (baseline)	0.82 acc / 278,532 tok	Evolved candidates	none — no search ran

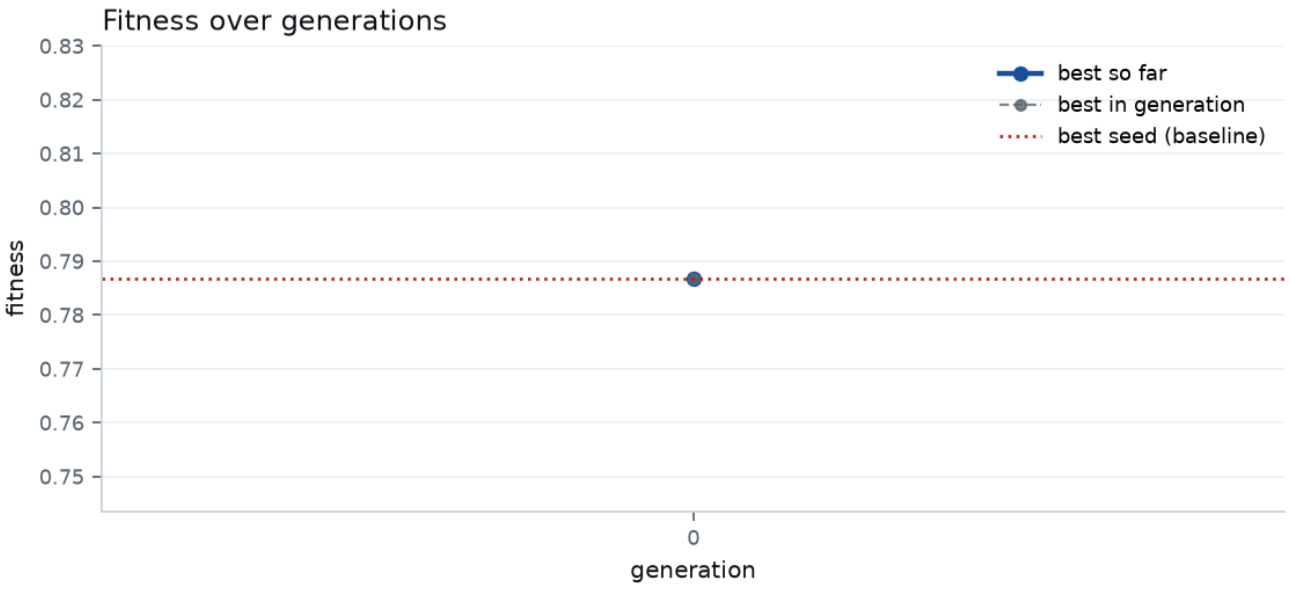
What the surrogate bought

No search was run, so the Predictor bought nothing here. The numbers below are measurement only.

The result

No search was run — this is a baseline measurement

The provider's free tier allows 500 requests per day per model, and a candidate costs roughly 165 across the task set. One model's daily allowance therefore buys three candidates: enough to measure the seed topologies against each other, not enough to evolve. The quota was exhausted before any generation completed. What follows is what was actually measured; no evolved candidate exists to compare against, and none is claimed.



Fitness over generations. The dotted line is the best seed — the network a one-shot designer would have handed you.

Method

Four phases, repeated. Phase C is the point.

```
Phase A   seed population evaluated for real      -->  (genome, fitness)
Phase B   train a Predictor on those pairs
Phase C   evolve thousands of candidates against it  --  zero LLM calls
Phase D   real-evaluate only the elite, feed back to B
```

The genome is neuro-san's own `agent_network_definition` with one addition, a per-agent model. neuro-san already overlays a per-agent `llm_config` over the network default and nobody tunes it, even though it is the main cost/quality knob in a multi-agent network.

Mutation operators

Operator	What it changes
add_agent / remove_agent	agent count; removal re-parents children
rewire	one edge, at fixed size
split_agent / merge_agents	granularity — the axis a designer told to ‘prefer the fewest agents’ never explores
toggle_search	which agents can read the corpus
reassign_model	per-agent model tier

Every mutant is checked — one top agent, a DAG, no unreachable agents, no dangling references, at least one agent able to search — and **discarded if it fails, never repaired**. A repaired mutant is a different mutant than the operator produced, and silently substituting one would make the search unable to learn what its own operators do.

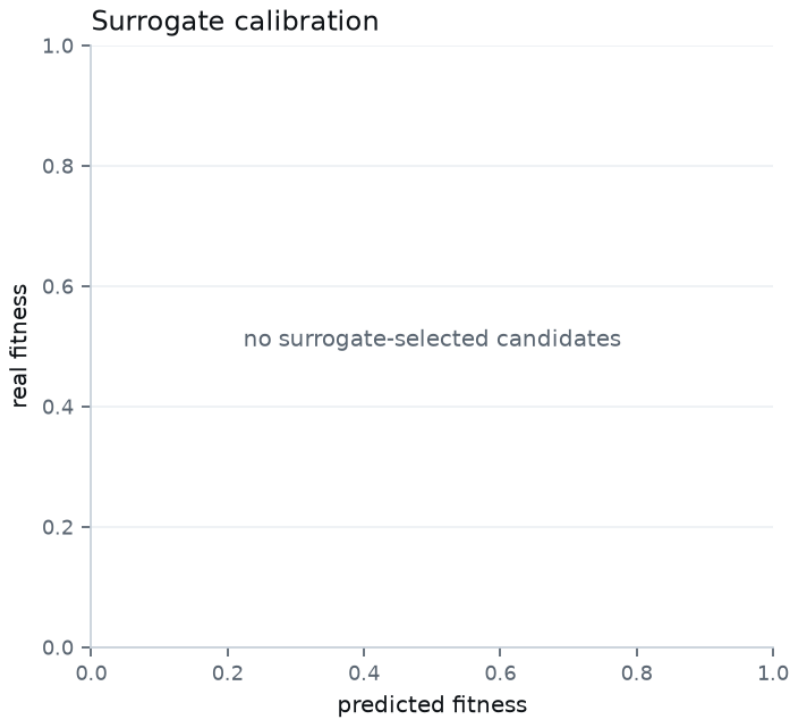
The evaluation world

A fictional logistics company: 24 depots, 40 contracts, 60 incidents, **124 documents**, and 17 questions spanning one to four hops. The world is synthetic so that no answer can be recalled from pretraining — fitness measures the topology, not the model's memory. Corpus and ground truth come from one seeded data model, so every answer is correct by construction. Retrieval is deterministic term overlap returning three documents: fitness must be a property of the network, not of a retrieval layer that drifts between generations.

The Predictor

Whether the surrogate earned its place.

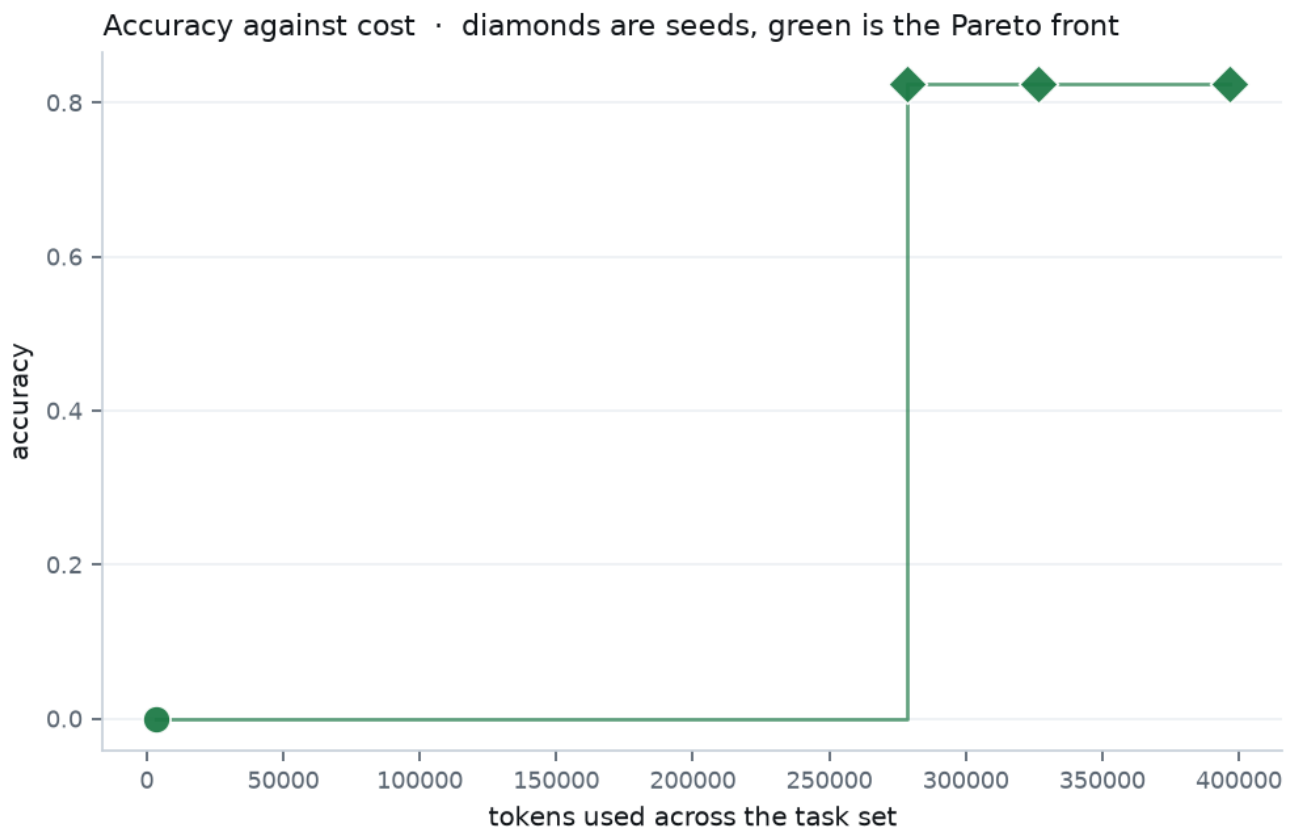
A surrogate trained on tens of samples is weak, and this one is trained on tens of samples. Its job is not to be right — it is to *rank*, well enough that the elite it selects are worth paying to run. The cross-validated rank correlation is published above whatever it says.



Predicted against real fitness for the candidates the surrogate chose. Points on the dotted line would be perfect prediction.

Accuracy against cost

A single fitness number hides the trade-off. The front does not.



Every network evaluated. Diamonds are seeds, green is the non-dominated front.

Network	Origin	Accuracy	Tokens	Agents	Depth
459ac1a66d	seed:designer_shaped	0.82	278,532	4	2
bb3340f7f3	mutant	0.00	3,600	5	2
c3ee2e0c4a	seed:flat_pair	0.82	326,364	3	2
cbe128a617	seed:solo	0.82	396,378	1	1

Every network evaluated

The full record, in the order it was measured.

Gen	Origin	Network	Acc	Tokens	Agents	Fitness	Pred
0	seed:designer_shaped	459ac1a6	0.82	278,532	4	+0.7868	—
0	mutant	bb3340f7	0.00	3,600	5	-0.0115	—
0	seed:flat_pair	c3ee2e0c	0.82	326,364	3	+0.7842	—
0	seed:solo	cbe128a6	0.82	396,378	1	+0.7816	—

What measurement changed

Two findings that came from running it, not reasoning about it

Free-tier quota is per-model and wildly uneven. `gemini-3.6-flash` allows 20 requests per *day*, which cannot support an evolutionary run at all; `gemini-3.5-flash-lite` allows 15 per *minute*. Rate limiting became a correctness requirement, not a politeness one: a 429 returns as an agent error, the candidate scores zero, and the search learns that a good topology is bad.

The first task set had no headroom. A single agent with a single tool scored 1.00 on the original 13 questions. There was nothing for evolution to improve, so the corpus went from 40 documents to 124, retrieval narrowed from five results to three, and four-hop and aggregate shapes were added. An experiment whose baseline is already perfect measures nothing.

Limitations

- One task domain. A topology that wins at multi-hop retrieval need not win elsewhere.
- The surrogate is trained on tens of samples. It ranks; it does not price.
- The designer baseline is a faithful reconstruction of the shape `agent_network_designer` produces, not its literal output — the designer wires networks against its own toolbox and cannot see this corpus.
- Automated agent architecture search is an active field (ADAS, AFlow, GPTSwarm, MaAS). The idea is not new. What is absent from all of it is neuro-san, and what is absent from neuro-san is any fitness function.